Next Day Wildfire Spread Prediction

Harshil, Sushant, Priyanshu, Sachin, Piyush
CS & AI Department, Rishihood University

Abstract

We predict which areas an active wildfire will burn the next day, using the Next Day Wildfire Spread dataset (18 545 patches of U.S. fires at 1 km resolution), where only 1.1% of pixels burn on day $t+1$. We keep a per-pixel model and add 23 engineered spatial features, such as distance to the nearest fire and fire upwind. Tuned XGBoost reaches a test PR AUC of 0.257 (95% bootstrap interval over fire events 0.19–0.33), compared with 0.106 for persistence. The dataset authors report 0.225 for a random forest and 0.284 for a CNN, from a different training pipeline, so these are reference points rather than a controlled comparison. Ablation and learning curves show that the spatial features carry the signal: tuning changes the score by only about 0.01, and 20 times less training data barely changes it. The model finds fire near the existing front but misses spread beyond 5 km, which motivates a spatial deep model in Phase 2.

Index terms: wildfire spread, remote sensing, gradient boosting, class imbalance, precision-recall.

I. Introduction

Large wildfires destroy homes and ecosystems, and their smoke harms health far from the fire. Fire managers decide each day where to send crews and whom to warn, so they need to know where an active fire will be tomorrow. Physics-based spread models answer this question, but they need detailed fuel and weather inputs [1] that are often missing at scale. Machine learning offers a data-driven alternative [2].

We study next-day wildfire spread as defined by Huot et al. [3]: given 12 input channels on day t, predict for every pixel whether it is burning on day $t + 1$. Their Next Day Wildfire Spread dataset contains 18 545 patches of U.S. fires from 2012 to 2020, each covering 64×64 km at 1 km resolution with weather, terrain, vegetation, population and the day-t fire mask.

The task is hard for two reasons. First, fire is rare: only 1.1% of labelled training pixels burn on day $t + 1$, a 90:1 imbalance, so accuracy is meaningless and we report PR AUC. Second, the labels are noisy: clouds and smoke hide part of the fire mask, and these uncertain pixels must be left out of training and evaluation.

Our approach keeps a simple per-pixel model and moves the spatial reasoning into the features. We build a pixel table with every fire pixel and a 5:1 sample of no-fire pixels, add 23 engineered spatial features to the 12 raw channels, such as the distance to the nearest fire and fire upwind, and train XGBoost and LightGBM tuned on eval PR AUC (Fig. 1). Tuned XGBoost reaches a test PR AUC of 0.257 against 0.106 for persistence.

Our contributions are:

- a per-pixel pipeline with 23 engineered spatial features that reaches a test PR AUC of 0.257 (95% bootstrap interval over fire events 0.19–0.33), between the random forest (0.225) and the CNN (0.284) reported by Huot et al.;
- ablation, tuning and learning-curve results showing that the signal lies in the fire neighbourhood and that the model is limited by bias, not variance;
- an error and event-level analysis showing where the model fails: spread beyond 5 km and false alarms around dying fires;
- data findings that matter for any model on this dataset: the patches are stored south-up, the raw export contains impossible values, and the test split contains duplicate patches.

Section II reviews related work and Section III gives the theory behind our choices. Section IV describes the dataset and our exploratory analysis, Sections V and VI the method and the engineered features, and Section VII the results. Section VIII concludes and Section IX outlines Phase 2.

II. Related Work

We review the literature in four steps: the physics-based model that learned models aim to replace, machine learning in wildfire science, learned spread models on NDWS and related datasets, and the methods our pipeline is built on. Table I compares the spread studies, and Section II-F places our work among them.

A. Physics-based fire spread

Rothermel [4] developed the surface fire spread model that still underlies U.S. fire behaviour tools. Andrews [1] describes it as quasi-empirical. It combines a heat-balance

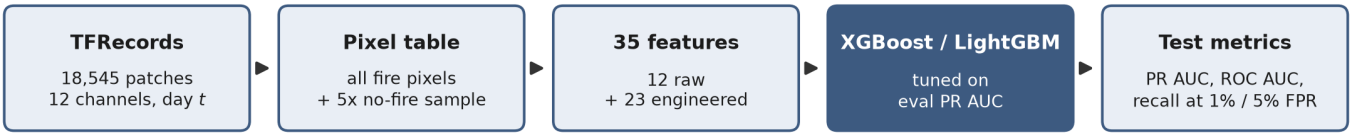


Fig. 1. Phase 1 pipeline. TFRecords are parsed into a pixel table with all fire pixels and a 5:1 sample of no-fire pixels, described by 35 features (12 raw and 23 engineered). XGBoost and LightGBM are tuned on eval PR AUC and scored once on the test split.

formulation with wind-tunnel experiments in artificial fuel beds and grassfire observations. The model computes the steady-state rate of spread of a fire front through a uniform fuel bed. Its inputs are fuel load, depth, surface-area-to-volume ratio, heat content, mineral content, moisture and moisture of extinction, together with midflame wind speed and slope. It was first used in the 1972 U.S. National Fire Danger Rating System (NFDRS) and is built into BehavePlus and FARSITE [1]. Relevance: the energy release component (erc) in our data is an NFDRS output. Our features use the same drivers (wind, slope, dryness), but leave the form of the relationship to the learner.

B. Machine learning in wildfire science

Jain et al. [2] review machine learning in wildfire science and management. They find 300 publications up to the end of 2019. Fire occurrence, susceptibility and risk is the largest domain (127 papers), and fire behaviour prediction, which includes spread, has 43. Random forests, MaxEnt, neural networks, decision trees, SVMs and genetic algorithms are the most common methods. Relevance: spread prediction is a small part of the field.

Sayad et al. [5] predict wildfire occurrence from three MODIS products (NDVI, land surface temperature, thermal anomalies) over central Canada. Their table has 804 rows (386 fire, 418 no fire), and an ANN and an SVM reach 98.32% accuracy. Relevance: with balanced classes, accuracy is informative. On our test split, 98.75% of pixels do not burn, so accuracy is not.

C. Learned spread models on other data

Hodges and Lattimer [6] train a deep convolutional inverse graphics network to reproduce simulated burn maps (Rothermel's equations and FARSITE) every 6 h up to 24 h. They report precision, sensitivity, F-measure and Chan-Vese similarity of 0.97, 0.92, 0.93 and 0.93 against the simulator, at a computational cost three orders of magnitude lower. Relevance: these scores measure agreement with a simulator, not with observed fires.

Radke et al. [7] (FireCast) predict 24 h spread from GeoMAC perimeters of Rocky Mountain fires since 2013, Landsat-8 imagery, elevation and weather, using a CNN on a window around each pixel. On the 2016 Beaver Creek fire they report 87.7% accuracy and 91.1% recall, but 3.6% precision and a 6.4% F-score. Relevance: when fire is rare, accuracy and recall can hide very low precision.

Gerard et al. [8] build WildfireSpreadTS, a multi-day dataset of 13 607 VIIRS images (375 m) of 607 U.S. fires from 2018 to 2021, scored by average precision over 12 year-based folds. Persistence reaches 0.193, logistic regression up to 0.288, a ResNet-18 U-Net 0.341, and a temporal attention U-Net (UTAE) with five input days 0.372. Relevance: a different sensor and split, but the same ordering: spatial and temporal models beat pixel models.

D. Learned spread models on NDWS

Huot et al. [3] introduce Next Day Wildfire Spread (NDWS): 18 545 fire events over the contiguous U.S. (2012–2020), each a 64×64 km patch at 1 km with 12 day-t inputs and the day-$(t{+}1)$ MODIS fire mask. Whole weeks are assigned to train, eval and test in an 8:1:1 ratio. On PR AUC, persistence scores 0.115, logistic regression 0.198, and a random forest of depth 15 scores 0.225; both pixel models see each pixel and its eight neighbours. A convolutional autoencoder that sees the whole patch reaches 0.284. Relevance: we use their data, split and metric.

Fitzgerald et al. [9] add attention blocks to a U-Net and train it with focal loss on random 32×32 crops, discarding crops with uncertain labels. The best model reaches F1 0.370; no PR AUC is reported. They also note that most next-day fire lies in the patch centre and that models learn to predict fire there. Relevance: cropped, filtered test sets give numbers that cannot be compared with full-patch results.

Xiao et al. [10] propose an attention U^2-Net and report F1 0.405. They re-split all 18 545 samples at random 8:1:1 instead of using the official week-based split, and fill uncertain labels with 0. Relevance: a random re-split can place samples of the same fire in training and test.

Andrianarivony and Akhloufi [11] (LinU-Mamba) use the official split and report F1 42.22 ± 0.29 over 10 runs, with uncertain labels clamped to 0 and no PR AUC. Retraining earlier models on the full split, they find much lower F1 than those papers reported on subsets (20.06 instead of 70.97 for a multi-kernel CNN [12]). Relevance:

evaluation protocol changes the numbers as much as the model does.

Zhou et al. [13] compare an autoencoder, ResNet, U-Net and Swin-UNet on 32×32 crops; the split is not stated. Swin-UNet reaches PR AUC 0.280 and U-Net 0.274, and an XGBoost baseline, whose setup is not described, reaches 0.06. Relevance: the only other tree model on NDWS we found; our per-pixel XGBoost reaches 0.257 with engineered spatial features.

Two recent preprints report PR AUC on full patches with uncertain labels excluded. Esparza et al. [14] use the official split, choose the threshold on training and eval data only, and give bootstrap intervals. Their re-run of the convolutional autoencoder reaches 0.327 and a U-Net 0.346, and their best ensemble 0.379. Han et al. [15] (FireSenseNet) report PR AUC 0.3435, but choose their F1 threshold on the test set. They also show that predicting the union of both days' fire and setting uncertain labels to 0 inflate F1 by 44–50%, as in [16]. Relevance: these are the closest comparisons to our protocol, but neither is peer-reviewed.

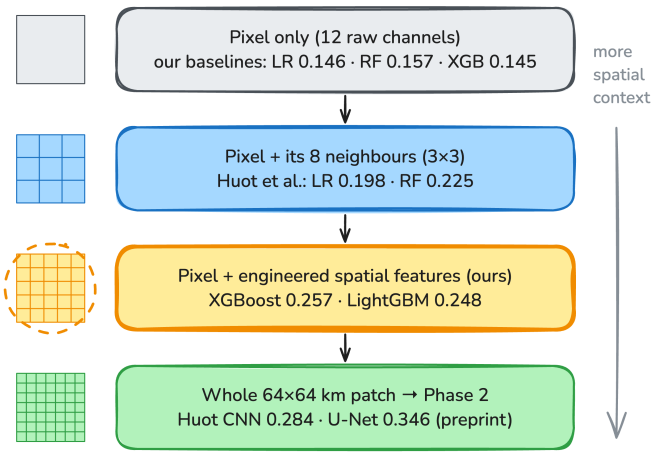


Fig. 2. NDWS models ordered by the spatial context they see, with test PR AUC. Raw-channel baselines are ours; the others are from [3], [14].

E. Methods we build on

Gradient-boosted trees. Friedman [17] frames boosting as gradient descent in function space, with regression trees as base learners. Chen and Guestrin [18] add a regularised objective and second-order split finding in XGBoost, our main model. Ke et al. [19] speed up histogram-based boosting with gradient-based sampling and feature bundling in LightGBM, our second model, which trains up to more than 20 times faster at similar accuracy. Breiman's random forests [20] average decorrelated trees; they are the strongest pixel model of Huot et al. Section III-A explains how these ideas set our hyperparameters.

Segmentation networks. Ronneberger et al. [21] propose U-Net, an encoder-decoder with skip connections for segmentation with few labelled images; it won the ISBI 2015 cell-tracking challenge on two datasets. It is the base of most NDWS models above and a candidate for our Phase 2.

Learning and evaluating under imbalance. Focal loss [22] down-weights easy examples, the alternative to class weighting discussed in Section III-B. Davis and Goadrich [23] prove that one curve dominates in ROC space if and only if it dominates in PR space and that PR curves must not be interpolated linearly, and Saito and Rehmsmeier [24] show that ROC plots can mislead on imbalanced data. Both support PR AUC as our metric (Section III-C).

Interpretation. SHAP [25] attributes a prediction to features with Shapley values, and TreeSHAP [26] computes them exactly for tree ensembles. Our Results section uses them to check that the model's dependence on fire distance, dryness and wind is physically sensible.

F. Where our work sits

In our project

Three points stand out from Table I and Fig. 2.

Protocols differ. Crops, the handling of uncertain labels, splits and test-set thresholds change the reported numbers. Only Huot et al. and Esparza et al. clearly use the official split with full patches and PR AUC, the protocol we follow.

Spatial features help trees. The NDWS work we found is almost entirely deep spatial models. Trees appear only as Huot et al.'s neighbour-window baselines (0.225) and as an XGBoost baseline at 0.06. Our per-pixel XGBoost with engineered spatial features reaches 0.257. That is well above the 0.06 baseline and above Huot et al.'s 0.225, although our 95% interval over fire events (0.188–0.325, Appendix B) also contains 0.225. The ablation traces the gain to the fire-neighbourhood features.

Spatial models are still ahead. Huot et al.'s CNN (0.284) and recent U-Nets (0.346 in a preprint) score higher. This gap is what motivates Phase 2: a spatial model that sees the whole patch and the shape of the fire front. Unlike the papers above, we checked whether the same location appears in more than one split (Section V): it does for 30% of test patches, but the ranking of models holds on unseen locations.

TABLE I
Spread studies at a glance. Only rows with the same data, split and metric are comparable; the protocol column gives the differences. Preprints are marked †.

Paper	Method	Data / protocol	Metric	Result
Hodges & Lattimer 2019 [6]	CNN (DCIGN)	simulated burn maps	F vs. simulator	0.93
Radke et al. 2019 [7]	CNN + weather	GeoMAC/Landsat-8, Rocky Mts.	Precision / recall	3.6% / 91.1%
Gerard et al. 2023 [8]	U-Net / UTAE	WildfireSpreadTS, VIIRS 375 m	AP	0.341 / 0.372
Huot et al. 2022 [3]	Persistence / LR / RF	NDWS official split; LR, RF use 3×3 window	PR AUC	0.115 / 0.198 / 0.225
Huot et al. 2022 [3]	Conv. autoencoder	NDWS official split	PR AUC	0.284
Zhou et al. 2025 [13]	U-Net / Swin-UNet / XGBoost	NDWS, 32×32 crops, split not stated	PR AUC	0.274 / 0.280 / 0.06
Esparza et al. 2026† [14]	Conv. AE / U-Net	NDWS official split, full patches	PR AUC	0.327 / 0.346
Han et al. 2026† [15]	Dual-branch CNN	NDWS 8:1:1, F1 threshold on test	PR AUC	0.3435
Fitzgerald et al. 2023 [9]	Attention U-Net	NDWS, 32×32 crops, uncertain crops dropped	F1	0.370
Xiao et al. 2024 [10]	Attention U^2-Net	NDWS random re-split, uncertain $\to 0$	F1	0.405
Andrianarivony & Akhloufi 2025 [11]	LinU-Mamba	NDWS official split, uncertain $\to 0$	F1	0.422
Ours	XGBoost, 35 features	NDWS official split, full patches	PR AUC	0.257

III. Theoretical Background

This section gives the theory behind four choices in our pipeline (Fig. 1): gradient-boosted trees, class weighting, PR AUC as the main metric, and why tuning stops helping. The main results are stated here, and the full derivations are given in Appendix A.

A. Gradient boosting and second-order splits

Gradient boosting [17] builds a model as a sum of regression trees, adding one tree f_m at a time and shrinking each by a learning rate η:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, f_m(\mathbf{x}). \tag{1}$$

The score F is a log-odds, and the predicted probability is $p = \sigma(F) = 1/(1+e^{-F})$. XGBoost [18] chooses each tree by minimising a loss plus a penalty on tree complexity, where T is the number of leaves and w_j the value of leaf j:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k} \Omega(f_k), \tag{2}$$

$$\Omega(f) = \gamma T + \tfrac{1}{2}\lambda \textstyle\sum_j w_j^2. \tag{3}$$

Second-order step. At step m the loss is expanded to second order around the current prediction. With gradients g_i and Hessians h_i of the loss, which for log loss are (Appendix A-A)

$$g_i = p_i - y_i, \qquad h_i = p_i(1-p_i), \tag{4}$$

and with G_j and H_j the sums of g_i and h_i over the rows in leaf j, the objective becomes one quadratic per leaf:

$$\mathcal{L}^{(m)} \approx \sum_{j=1}^{T} \Big[G_j w_j + \tfrac{1}{2}(H_j + \lambda)\, w_j^2 \Big] + \gamma T. \tag{5}$$

Leaf values and split gain. Setting the derivative with respect to w_j to zero, $G_j + (H_j + \lambda)\, w_j = 0$, gives the optimal leaf value and, substituted back, the lowest loss for a given tree structure:

$$\boxed{w_j^* = -\frac{G_j}{H_j + \lambda}}, \qquad \mathcal{L}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T. \tag{6}$$

Each leaf therefore takes one Newton step, the gradient divided by the curvature plus λ. Splitting a leaf into a left part L and a right part R changes $\mathcal{L}^*$ by

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma. \tag{7}$$

Trees are grown greedily, and a split is kept only if its gain is positive (Fig. 3). So γ is the minimum loss reduction that pays for a new leaf, and λ shrinks leaf values toward zero, most strongly in leaves with a small Hessian sum.

LightGBM. LightGBM [19] minimises the same kind of objective but differs in four ways. It bins each feature into a histogram, so split search scans bins instead of

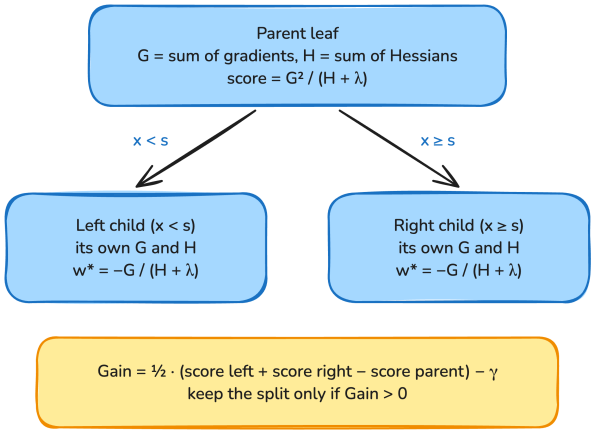


Fig. 3. One split in XGBoost. Each child gets the value w^* of (6); the split is kept only if the gain (7) is positive.

TABLE II
Boosting symbols and our selected values (reports/best_params.json).

Symbol	Meaning	XGBoost	LightGBM
λ	L2 penalty on leaf values	7.22	0.28
γ	minimum gain per split	1	1
η	learning rate	0.026	0.058
$\min H_j$	minimum leaf size	$H_j \geq 1$	500 rows
-	tree size	depth 6	15 leaves
-	rows per tree	0.73	0.98
-	features per tree	0.63	0.88
w	weight on fire rows	5.0	5.0
M	trees after early stopping	216	294

sorted values. It grows trees leaf-wise, always splitting the leaf with the largest loss reduction, so tree size is set by num_leaves rather than depth. It can sample rows by gradient size (GOSS). It bundles sparse features that are rarely non-zero together (EFB).

Early stopping. Each added tree lowers the training loss, so the number of trees sets how closely the ensemble fits the training table. Stopping when the held-out score no longer improves chooses that number on data the trees were not fitted to, and in that way limits overfitting.

In our project

Table II maps each symbol to its parameter and selected value (search space in Table VI). In XGBoost, min_child_weight is the smallest H_j allowed in a child. By (4) a no-fire row has $h \leq 0.25$ and a fire row, weighted by 5, has $h \leq 1.25$, so our value of 1 is a weak constraint. The other two penalties are strong: $\gamma = 1$ is the largest value searched, and $\lambda = 7.22$ lies near the top of $[0.1, 10]$. XGBoost uses histogram binning (tree_method=hist). LightGBM grows leaf-wise with 15 leaves and uses plain bagging, not GOSS. Both models stop after 50 rounds without improvement in eval PR AUC, keeping 216 and 294 of at most 1 000 trees. We stop on PR AUC rather than log loss because the 1:5 training table shifts the probabilities (Section III-B). On log loss, our LightGBM runs stopped after a single tree (src/train.py).

B. Class imbalance and class weighting

Next-day fire covers 1.10% of labelled training pixels, about 90 no-fire pixels per fire pixel. Left uncorrected, no-fire rows dominate G_j and H_j in every leaf. The two standard corrections are to subsample negatives and to weight positives.

Weighting acts like duplication. With weight w on positives, the log loss and the gradient and Hessian of a positive row become (Appendix A-A)

$$l_w = -\big[\, w\, y \log p + (1-y)\log(1-p)\,\big], \tag{8}$$

$$g_i = w\,(p_i - 1), \qquad h_i = w\,p_i(1-p_i). \tag{9}$$

Both are w times their unweighted values. Trees see the data only through G and H, so a weighted positive acts exactly like w copies of the same row.

Both corrections shift the probabilities. Keeping a share r of the negatives and weighting positives by w changes the class prior seen by the loss but not the feature distribution within each class. By Bayes' rule (Appendix A-C), the probability p' learned in training and the true probability p differ by a constant factor in their odds:

$$\frac{p'}{1-p'} = \frac{w}{r} \cdot \frac{p}{1-p} \tag{10}$$

Solving for p maps a model score back to the true scale:

$$p = \frac{p'}{p' + (w/r)(1-p')}. \tag{11}$$

The map is strictly increasing, so the order of pixels is unchanged. Ranking metrics (PR AUC, ROC AUC, recall at a fixed FPR) are unaffected, but a fixed threshold must be chosen on data with the true class ratio.

Focal loss. Lin et al. [22] reshape the loss instead of the data. With $p_t = p$ for positives and $1-p$ for negatives,

$$\mathrm{FL}(p_t) = -\alpha_t (1-p_t)^{\gamma_f} \log p_t, \tag{12}$$

where $\gamma_f = 2$ and $\alpha = 0.25$ worked best in their experiments. The factor $(1-p_t)^{\gamma_f}$ is close to zero for well-

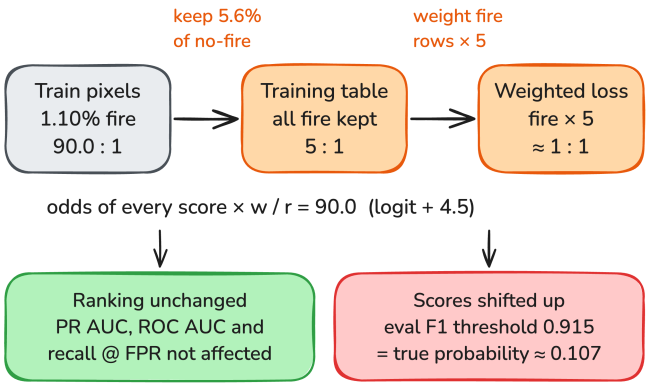


Fig. 4. Subsampling and weighting turn a 90:1 problem into a balanced one and shift every score, by (10).

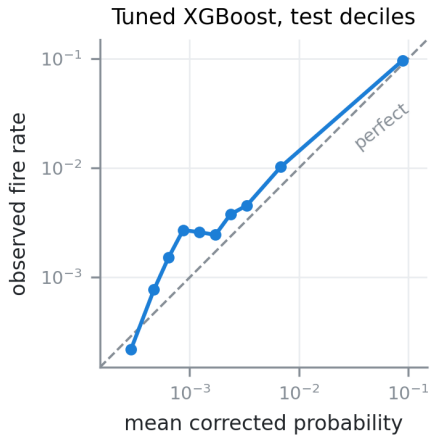


Fig. 5. Calibration of the tuned XGBoost on test after undoing the prior shift with (11). Each point is one tenth of the test pixels, ordered by score; log scales.

classified rows, so easy negatives contribute little. With $\gamma_f = 0$ the loss reduces to (8).

In our project

The training table keeps every fire pixel and 5 no-fire pixels per fire pixel, so $r = 5/90.0 = 0.056$, and fire rows get $w = 5$ (scale_pos_weight). The odds are therefore multiplied by $w/r = 90.0$, and the model trains as if the classes were balanced (Fig. 4). This explains the high F1 threshold of 0.915 chosen on eval: by (11) it corresponds to a probability of 0.107 on the true scale, while a score of 0.5 corresponds to 0.011, the training fire rate. We checked the correction on test with a retrained copy of the model (Appendix B). After (11), the mean probability is 0.0107 against an observed fire rate of 0.0125, and in the top score decile 0.089 against 0.097 (Fig. 5). The corrected values are therefore close to true risk and slightly low. Most of the overall gap matches the higher fire rate of test (1.25% against 1.10% in train); below a corrected value of about 0.002 they are up to about 3 times too low.
We did not use focal loss, because the boosting libraries would need it as a custom objective and weighting already balances the loss. It is a candidate for Phase 2, where a segmentation network computes its loss on every pixel of a patch (Huot et al. use a weight of 3 on fire for their CNN [3]).

C. PR AUC versus ROC AUC under imbalance

With P fire and N no-fire pixels, and TP and FP the true and false positives at a threshold,

$$\mathrm{TPR} = \frac{\mathrm{TP}}{P}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{N}, \tag{13}$$

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}. \tag{14}$$

The ROC curve plots TPR (recall) against FPR, and the PR curve plots precision against recall. Writing precision in terms of the two rates and the prevalence $\pi = P/(P+N)$ (Appendix A-D) gives

$$\mathrm{Precision} = \frac{\pi\,\mathrm{TPR}}{\pi\,\mathrm{TPR} + (1-\pi)\,\mathrm{FPR}} \tag{15}$$

TPR and FPR are each computed within one class, so the ROC curve does not depend on π. Precision does: when π is small, $(1-\pi)\,\mathrm{FPR}$ dominates the denominator even at a small FPR.

A random classifier has ROC AUC 0.5 at any imbalance, but it also has TPR = FPR, so by (15) its precision is π everywhere and its PR AUC equals the prevalence [24]. We compute PR AUC as average precision, $\mathrm{AP} = \sum_k (R_k - R_{k-1})\,P_k$, a step sum over thresholds with recall R_k and precision P_k. Davis and Goadrich [23] prove that a curve dominates in ROC space if and only if it dominates in PR space, and that linear interpolation is wrong in PR space, which is why AP uses a step sum. Saito and Rehmsmeier [24] show that on imbalanced data the ROC plot can be deceptive, while the PR plot is not.

In our project

Fire is 1.25% of labelled test pixels, so a random model scores PR AUC 0.0125. Our tuned XGBoost scores ROC AUC 0.906 but PR AUC 0.257 (Table VIII). Fig. 6 shows why the two differ. At 1% FPR the model finds about 32 800 of the 84 331 fire pixels but raises about 66 400 false alarms, so precision is 0.33. At 5% FPR it finds about 56 900 against 332 200 false alarms, a precision of 0.15. Recall at 1% and 5% FPR therefore fixes the false-alarm budget an operator

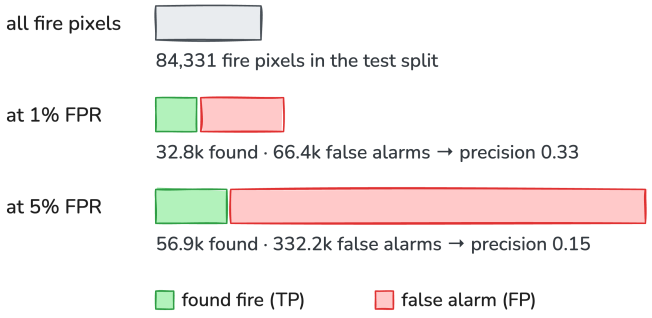


Fig. 6. Tuned XGBoost on the test split at 1% and 5% FPR. Box widths are proportional to pixel counts.

would have to check. Precision, recall and F1 use the F1-optimal threshold from eval, applied unchanged to test. Choosing it on test would overstate F1 (0.358 against 0.332).

D. Bias-variance trade-off of tree ensembles

For squared loss, the expected error of a model $\hat{f}$ trained on a random training set splits into three parts, where f is the true function and σ^2 the noise variance:

$$\mathbb{E}\big[(y-\hat{f})^2\big] = \underbrace{\big(\mathbb{E}\hat{f}-f\big)^2}_{\text{bias}^2} + \underbrace{\operatorname{Var}\hat{f}}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}. \qquad (16)$$

For log loss the split is not exactly additive, but the same reasoning applies. Bias comes from a model or input representation that is too restricted, and variance from sensitivity to the training sample.

Averaging trees. For B trees, each with variance σ^2 and pairwise correlation ρ, the variance of their average is (Appendix A-E)

$$\operatorname{Var}\Big(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b\Big) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 \qquad (17)$$

More trees remove only the second term, so the shared variance $\rho\sigma^2$ stays. Random forests [20] lower ρ with bootstrap samples and a random subset of features at each split.

Boosting. Boosting starts from small, high-bias trees and lowers the bias step by step. Its variance is held back by shrinkage, early stopping, tree size, minimum leaf size, the penalties λ and γ, and row and column subsampling, which by (17) also make the trees less alike.

In our project

Across 20 random XGBoost configurations (depth 4 to 10, learning rate 0.023 to 0.28), eval PR AUC stayed between 0.196 and 0.207, and the 20 LightGBM

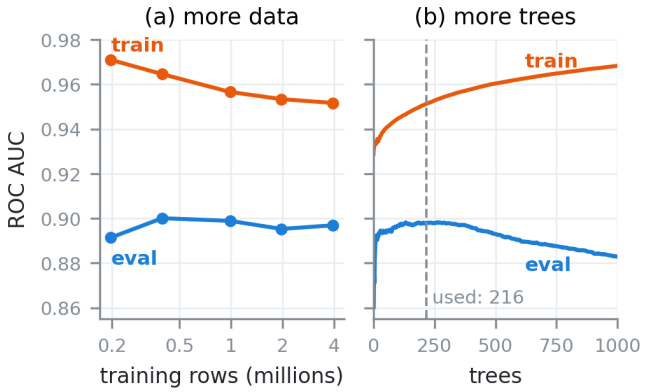


Fig. 7. Tuned XGBoost, ROC AUC on training rows and on eval. (a) Trained on 5–100% of the training patches. (b) Trees added without early stopping; dashed: the 216 trees used. ROC AUC is shown because the training table is subsampled 1:5, which changes PR AUC but not ROC AUC (Section III-C).

configurations between 0.189 and 0.203. The settings that trade bias against variance therefore move the score by only about 0.01. Changing the inputs moves it far more: with the same hyperparameters, removing the fire-neighbourhood features lowers LightGBM's test PR AUC from 0.248 to 0.048 (Table IV). Learning curves (Fig. 7) show where the remaining error lies. Trained on 5% of the training patches (0.2M rows) instead of all of them (3.9M rows), the tuned XGBoost reaches almost the same eval PR AUC (0.204 against 0.207), and eval ROC AUC stays between 0.891 and 0.900. More data narrows the train-eval gap in ROC AUC from 0.080 to 0.055 only by lowering the training score. More trees do the opposite: training ROC AUC keeps rising while eval peaks at 0.898 near 270 trees and falls to 0.883 at 1000, the variance that early stopping (216 trees) avoids. An eval score that does not move with 20 times more data means the model is not limited by variance. What remains is bias from the input representation, since each pixel sees the patch only through fixed-radius summaries, and noise the inputs cannot explain, such as which fires will die down. This is why Phase 2 moves to a spatial model rather than to more data or tuning.

IV. Dataset and Exploratory Analysis

A. Dataset

We use the Next Day Wildfire Spread dataset [3]: 18 545 samples of active fires across the contiguous United States from 2012 to 2020. Each sample is a 64×64 km patch at 1 km resolution with 12 input channels on day t (elevation from SRTM; wind direction and speed, minimum and

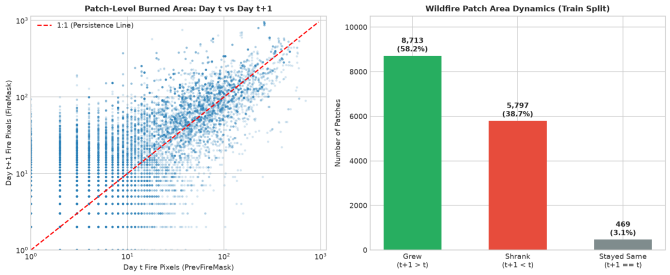


Fig. 8. Burned area per training patch on day t vs. day $t+1$ (left, log scale, dashed line = persistence) and the share of patches that grow, shrink or stay the same (right).

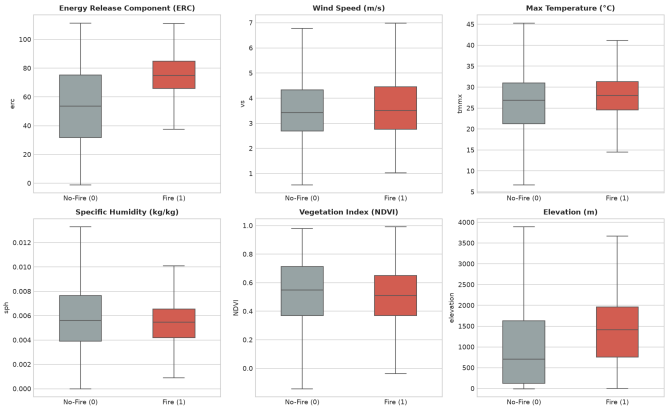


Fig. 9. Raw channel distributions for next-day fire vs. no-fire pixels (balanced 50k/50k training sample, outliers hidden).

maximum temperature, specific humidity, precipitation and ERC from GRIDMET; the Palmer drought index from GRIDMET Drought; NDVI from VIIRS; population density from GPWv4; and the MODIS active-fire mask) and the MODIS fire mask on day $t+1$ as the label. Fires more than 10 km apart are treated as separate events, and every sample contains fire on day t. Calendar weeks are assigned at random to train, eval and test in an 8:1:1 ratio with a one-day buffer, giving 14 979 / 1 877 / 1 689 patches. Table III lists every channel with its unit, source, native resolution and value range.

B. Labels and class imbalance

Fire masks use three values: fire (1), no fire (0) and uncertain (-1, e.g. cloud or smoke). Uncertain pixels make up 1.1–1.4% of day-t masks and 2.3–2.8% of day-t+1 masks, with similar shares in every split, so ignoring them in training and evaluation introduces no shift between splits. Next-day fire covers only 1.10% of labelled training pixels (90:1), 1.37% on eval and 1.25% on test; an all-zero predictor would exceed 98.7% accuracy, which is why we report PR AUC. Across the 14 979 training patches, 58.2% of fires grow from day t to t+1, 38.7% shrink and 3.1% keep the same size (Fig. 8), matching the 58%/39% reported by Huot et al. Shrinking fires make persistence over-predict, and many small day-t fires gain far more pixels on day $t+1$.

C. Data quality

The raw export contains physically impossible values: wind direction outside $[0°, 360°]$ in 0.23% of training pixels (from $-505\,870°$ to $37\,736°$), temperatures from -445 K to 1 230 K in 0.36% of pixels, negative precipitation in 9.8% of pixels, and a few negative wind speed, humidity and ERC values. These are corrected as described in the Methodology section (Table V). Negative elevation (down to -45 m) and negative NDVI (water, snow) are real and left unchanged. NDVI is stored multiplied by 10^4.

D. Channel relationships and marginal signal

Minimum and maximum temperature are strongly correlated (Spearman $\rho = 0.89$), humidity follows temperature ($\rho = 0.70$ with tmmn), and elevation correlates with ERC (0.66) and negatively with humidity (-0.48) and population (-0.47). The day-t fire mask has near-zero rank correlation with every raw channel ($|\rho| \leq 0.09$). Comparing next-day fire and no-fire pixels (balanced sample, Fig. 9), ERC (median 75 vs. 54) and elevation (1 414 vs. 709 m) separate the classes most clearly, maximum temperature and NDVI shift slightly (28.0 vs. 26.8 °C; 0.51 vs. 0.55), and wind speed and specific humidity barely differ at the median.

E. From EDA to features

Three findings shaped the pipeline. (i) A pixel's own weather says little about whether it burns, while burning depends strongly on nearby fire, so most engineered features describe the fire neighbourhood (Table VII). (ii) Measuring each next-day fire pixel's distance to the nearest day-t fire shows that 40.4% lie more than 5 km away and 26.1% more than 10 km away; we therefore added an upwind fire feature at 8 km (upwind_fire_k8), which ranks 5th of 35 features by gain in the tuned XGBoost. (iii) Wind direction is circular, so out-of-range values are wrapped rather than clipped. We did not clip negative NDVI, because water and snow are genuine, informative non-fuel.

F. Feature-group ablation

To measure what each feature group contributes, we retrain the tuned LightGBM with identical hyperparameters while removing one group at a time, plus a run with the fire-neighbourhood group only (Table IV, Fig. 10). Each run chooses its F1 threshold on eval and is scored on

TABLE III
Input channels and label. Value range is the 1st–99th percentile over the training split.

Name	Description	Unit	Source	Native res.	Range (p1–p99)
elevation	Terrain height	m	SRTM	30 m	2 to 2 954
th	Wind direction (from)	degrees	GRIDMET	4 km	31 to 331[a]
vs	Wind speed	m/s	GRIDMET	4 km	1.39 to 7.54
tmmn	Minimum temperature	K	GRIDMET	4 km	264.7 to 297.1
tmmx	Maximum temperature	K	GRIDMET	4 km	277.6 to 312.7
sph	Specific humidity	kg/kg	GRIDMET	4 km	0.0012 to 0.0178
pr	Precipitation (daily total)	mm	GRIDMET	4 km	−0.02 to 7.52
pdsi	Palmer drought index	index	GRIDMET Drought	4 km	−5.14 to 5.25
NDVI	Vegetation index	index $\times 10^4$	VIIRS VNP13A1	500 m	0 to 8 936
population	Population density	people/km^2	GPWv4	1 km	0 to 851
erc	Energy release component	index	GRIDMET (NFDRS)	4 km	11.3 to 104.1
PrevFireMask	Active fire on day t	class	MODIS MOD14A1	1 km	$\{-1, 0, 1\}$[b]
FireMask (label)	Active fire on day t+1	class	MODIS MOD14A1	1 km	$\{-1, 0, 1\}$[b]

Percentiles are used instead of min/max because the raw export contains artefacts (e.g. wind direction down to −505 870°, temperatures up to 1 230 K); observed extremes and their handling are listed in reports/data_quality.md and Table V. [a]Wrapped with θ mod 360 in the features (32 to 329 after wrapping). [b]1 = fire, 0 = no fire, −1 = uncertain (cloud or smoke); −1 label pixels are ignored in training and metrics.

TABLE IV
Feature-group ablation (LightGBM, test split, F1 at eval-chosen threshold).

Run	#feat.	PR AUC	Δ	F1
All features	35	0.248	–	0.331
No weather	21	0.248	−0.001	0.334
No vegetation	31	0.240	−0.009	0.333
No terrain	30	0.251	+0.003	0.340
No fire neighbourhood	25	0.048	−0.200	0.093
No human	33	0.251	+0.002	0.336
Fire neighbourhood only	10	0.218	−0.031	0.334

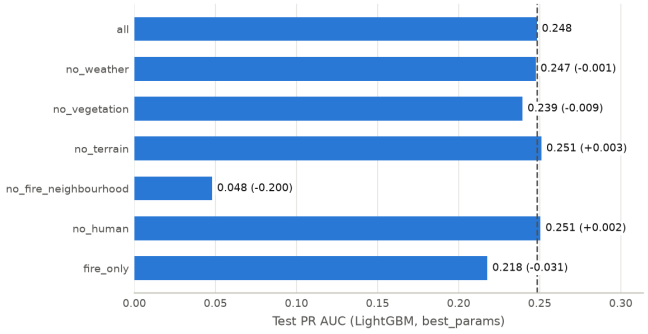


Fig. 10. Test PR AUC per ablation run; the dashed line marks the model with all features.

test. Removing the fire-neighbourhood features collapses PR AUC from 0.248 to 0.048, mirroring Huot et al., where removing the previous fire mask cuts PR AUC from 28.4% to 6.8%. Among the environmental groups, vegetation matters most (−0.009), again in line with Huot et al., who found vegetation the most useful variable alongside the fire mask. Removing weather costs only 0.001, and removing terrain or population changes PR AUC by +0.003 and +0.002; differences this small are within run-to-run noise and suggest these groups overlap with the remaining features. The fire-neighbourhood features alone reach 0.218, so the environmental variables add 0.031 (14%) on top of fire location. Because the upwind and alignment features use wind direction, the "no weather" run still sees some wind information.

V. Methodology

A. Problem formulation

Each sample of the Next Day Wildfire Spread dataset [3] is a 64×64 patch at 1 km resolution with 12 input channels observed on day t (11 environmental channels plus the active fire mask PrevFireMask) and a target fire mask FireMask on day t+1. We treat the task as per-pixel binary classification: for every pixel, predict the probability that it is burning on day $t+1$ using only information available on day t. Pixels with label −1 (unknown, e.g. cloud cover) are excluded from training and from every metric.

B. Data splits

We use the official train/eval/test split released with the dataset (14 979 / 1 877 / 1 689 patches). Huot et al. assign every calendar week between 2012 and 2020 at random to train, eval or test in an 8:1:1 ratio and leave a one-day buffer between weeks, so the splits are separated in time and a fire's day-t/day-t+1 pair never straddles two splits. All features are computed inside a single patch, so no feature uses pixels from another sample. The week-based split separates samples in time, but it does not guarantee spatial or fire-event independence: a long-lived fire can appear in weeks assigned to different splits. We checked this on the elevation grid: 503 of the 1 689 test patches

TABLE V
Physical clipping of raw channels (train split shares of affected pixels).

Channel	Bounds	Affected pixels
vs (wind speed)	≥ 0 m/s	$< 0.01\%$
tmmn, tmmx	$[253.15, 330]$ K	0.36%
sph	≥ 0	$< 0.01\%$
pr	≥ 0 mm	9.78%
erc	≥ 0	0.03%

(30%) cover exactly the same 64×64 location as a training patch, 38 of them with the same day-t fire mask, but none is an identical sample. These patches are easier for every model, persistence included (0.151 against 0.097), and the ranking is unchanged on new locations alone (tuned XGBoost 0.243, LightGBM 0.234, raw random forest 0.145, persistence 0.097). The eval split is used only for early stopping and hyperparameter selection; the test split is scored once per final model.

C. Preprocessing

The TFRecords are parsed directly from the protobuf wire format (no TensorFlow dependency) into arrays of shape $(N, 12, 64, 64)$. The raw export contains physically impossible values, so before feature construction each affected channel is clipped to fixed physical bounds (Table V). Wind direction is circular, so instead of clipping it is wrapped (θ mod 360; 0.23% of train pixels lie outside $[0, 360)$). The dataset does not state the patch orientation, so we verified it from the data: the arrays are stored south-up (the row index grows northward). Steep slopes facing the last row are clearly greener than those facing the first row (mean NDVI 0.596 vs. 0.555, while east- and west-facing slopes differ by only 0.008), as expected for pole-facing slopes in the Northern Hemisphere, and only this orientation makes next-day fire more frequent downwind than upwind in every wind-speed band (from $1.1\times$ in calm wind to $3.1\times$ at 8 m/s or more, 5 km from the fire; Results section). All direction-dependent features (upwind fire, wind alignment, aspect) use this orientation. The bounds are constants, not statistics fitted on the data, so clipping introduces no leakage. No normalisation is applied for the tree models, which are invariant to monotone rescaling of individual features.

D. Pixel tables and class imbalance

Fire pixels are rare: on the training split only 1.1% of labelled pixels burn on day $t + 1$ (a 90:1 imbalance). Each patch is flattened into one row per pixel. For the training table we keep every fire pixel and an independent random sample of non-fire pixels with probability chosen to give a 5:1 non-fire to fire ratio (fixed seed), giving 3.95 M rows. Uniform negative subsampling shifts the predicted base rate but not the ranking of pixels, so ranking metrics on the untouched eval and test splits remain valid. The remaining imbalance is handled by class weighting computed from the training table only: scale_pos_weight $= N_{\text{neg}}/N_{\text{pos}} = 5$. Eval (7.69 M pixels) and test (6.92 M pixels) keep every pixel in patch order, so predictions reshape directly into 64×64 probability maps.

TABLE VI
Random search spaces (20 configurations each).

Model	Hyperparameter	Range
XGBoost	max_depth	{4, 6, 8, 10}
	min_child_weight	{1, 5, 20, 100}
	gamma	{0, 0.1, 1}
LightGBM	num_leaves	{15, 31, 63, 127, 255}
	min_child_samples	{20, 100, 500, 2000}
	min_split_gain	{0, 0.1, 1}
Both	learning_rate	log-uniform $[0.02, 0.3]$
	subsample	uniform $[0.6, 1]$
	colsample_bytree	uniform $[0.5, 1]$
	reg_lambda	log-uniform $[0.1, 10]$

E. Models and tuning

We train XGBoost [18] and LightGBM [19] on all 35 features (12 raw plus 23 engineered). For each family we run a random search of 20 configurations over the space in Table VI. Every configuration grows up to 1 000 trees with early stopping (patience 50) on a fixed random sample of 1 M labelled eval pixels, and is then scored by PR AUC on the full eval split. The best configuration per family is evaluated once on test.

Eval PR AUC varied little across configurations (XGBoost 0.196–0.207, LightGBM 0.189–0.203), so the results are not sensitive to the exact hyperparameters. The selected XGBoost uses depth 6, learning rate 0.026 and 216 trees; the selected LightGBM uses 15 leaves, at least 500 samples per leaf, learning rate 0.058 and 294 trees. Full configurations are in reports/best_params.json.

F. Evaluation

Accuracy is uninformative at 1% prevalence, so every model is scored with the same function on the same pixels: PR AUC (average precision), ROC AUC, and recall at 1% and 5% false positive rate, which need no decision threshold. Precision, recall and F1 are reported at the threshold that maximises F1 on the eval split, applied unchanged to test; choosing it on test would overstate F1. The test-optimal F1 is kept only as an upper bound.

VI. Feature Engineering

Raw channels describe each pixel in isolation, but fire spread depends on where the fire is, which way the wind blows relative to it, how dry the air and fuel are, and

the local terrain. We engineer 23 features in five groups (Table VII); the groups are reused for the ablation study. Unless stated, local statistics use a $k \times k$ window with replicated edges, PrevFireMask $= -1$ is treated as no fire, and wind direction θ is the direction the wind blows from (degrees clockwise from north).

Fire neighbourhood. Spread happens at the fire front, so we measure burning density around each pixel at three scales, the distance to the nearest burning pixel, and the fraction of the patch already burning. Three wind-aware features look for fire $k \in \{2, 4, 8\}$ pixels upwind (8 because 40% of next-day fire pixels lie more than 5 km from day-t fire, per our EDA), and an alignment score is the cosine between the upwind direction and the direction to the nearest fire (+1 when the wind pushes the nearest fire toward the pixel).

Weather. Wind direction is circular, so it is replaced by u/v components. Specific humidity is converted to vapour pressure $e = qp/(0.622 + 0.378q)$, with surface pressure p from elevation (FAO-56). Saturation vapour pressure at the daily maximum temperature, $e_s = 0.6108 \exp\big(17.27T/(T + 237.3)\big)$ kPa, gives the vapour pressure deficit $e_s - e$ and minimum relative humidity e/e_s, both direct measures of atmospheric drying. We also add diurnal temperature range and the interaction erc×wind speed.

Vegetation, terrain, human. NDVI (stored $\times 10^4$) is summarised by its local mean, standard deviation and anomaly. Terrain adds slope and aspect from the elevation gradient and elevation relative to the 9×9 neighbourhood (ridges vs. valleys). Population is smoothed over 9×9 pixels as a proxy for settlement proximity and suppression effort.

Engineered features consistently carry more signal than the raw channels they are built from: dist_to_fire reaches a univariate AUC of 0.866 against 0.593 for the raw fire mask, and min_rel_humidity 0.705 against 0.565 for specific humidity. fire_wind_alignment has no marginal signal (0.501) because it is only meaningful close to an active fire, and gain importance in the tuned XGBoost shows the model barely uses it (23rd of 35 features, 0.5% of total gain). The fire-neighbourhood group dominates instead: dist_to_fire (32%), prev_fire_n9 (23%) and prev_fire_n5 (13%). The three upwind features rank 4th, 5th and 7th and together hold 13.5% of gain; upwind_fire_k8, added after the EDA showed long-range spread, ranks 5th (3.9%). With the patch orientation corrected, their combined share rose from 9.4% to 13.5%, so the model uses wind direction more once upwind is computed on the correct side. The raw PrevFireMask ranks only 32nd because the engineered neighbourhood features already contain its information.

VII. Results

All models are scored on the same 6 727 699 labelled test pixels (1 689 patches); 1.25% of them burn on day $t + 1$, about one pixel in 80. Precision, recall and F1 use the threshold that maximises F1 on the eval split; PR AUC, ROC AUC and recall at a fixed false positive rate (FPR) need no threshold.

A. Overall performance

The tuned XGBoost is the best model on every threshold-free metric (Table VIII): PR AUC 0.257 against 0.145–0.157 for the raw-channel models and 0.106 for persistence. LightGBM is close behind (0.248), so the features matter far more than the boosting library; the ablation (Table IV) traces the gain to the fire-neighbourhood group. At the eval threshold the tuned models find 52–53% of next-day fire at 24% precision, about twice the recall of the baselines (26–27%), and reach the best F1 (0.332); a threshold tuned on test would give only 0.358, so the eval threshold transfers well. Our persistence (0.106) is close to the paper's (0.115); the tuned XGBoost is above their neighbour-feature random forest (0.225) but below their CNN (0.284). Both values lie inside our 95% bootstrap interval over fire events (0.188–0.325, Appendix B), so neither difference is significant on test-sampling grounds. The same check shows that the gap to LightGBM is small but consistent (0.007, higher in 99% of paired resamples), and the gain over the raw-channel models clear (0.10, interval 0.09–0.12).

The tuned models stay above every baseline along the whole PR curve (Fig. 12). The raw-channel models rank already-burning pixels first and have little signal for new fire, so their operating points sit on top of persistence. ROC AUC is 0.795–0.906 for every learned model while PR AUC stays at or below 0.257; Section III-C explains why PR AUC is the more informative metric at this prevalence.

B. Probability maps

The predicted probability is a smooth halo around the day-t fire (Fig. 11), following its shape and stretched slightly toward where the wind blows (Fig. 11d; patch 33 in Fig. 16). After thresholding, the prediction is roughly the day-t fire grown by a few kilometres. It catches fire that stays or creeps outward (Fig. 11a; also patches 856 and 1432), paints a false-positive ring around small fires that shrink (Fig. 11b, patch 515: 48 to 29 burning pixels, 114 false positives), and misses fire that appears away from the day-t fire (Fig. 11c): in patches 120, 443, 1071 and 1369 not one next-day fire pixel is found. Unknown pixels (grey)

TABLE VII
Engineered features. AUC: univariate ROC AUC on the eval split, $\max(a, 1-a)$, sign gives direction of association. Raw channel shown for comparison.

Group	Feature	Definition	AUC
Fire	PrevFireMask (raw)	fire on day t	0.593 (+)
	prev_fire_n3/n5/n9	mean of fire indicator in 3/5/9 window	0.668 / 0.717 / 0.772 (+)
	patch_fire_frac	fraction of patch burning	0.759 (+)
	dist_to_fire	Euclidean distance to nearest fire (px $\approx$ km), $64\sqrt{2}$ if none	0.866 (−)
	upwind_fire_k2/k4/k8	prev_fire_n3 at the pixel 2/4/8 px upwind	0.649 / 0.615 / 0.586 (+)
	fire_wind_alignment	cos(upwind direction, direction to nearest fire)	0.501
Weather	sph (raw)	specific humidity	0.565 (−)
	wind_u, wind_v	$-v_s \sin\theta$, $-v_s \cos\theta$	0.522 / 0.518
	temp_range	tmmx − tmmn	0.602 (+)
	vpd	$e_s(T_{\max}) - e$	0.685 (+)
	min_rel_humidity	$e/e_s(T_{\max})$	0.705 (−)
	erc_x_wind	erc $\times v_s$	0.679 (+)
Vegetation	NDVI (raw)	NDVI $\times 10^4$	0.605 (−)
	ndvi_n5_mean/std	local mean / std of NDVI, 5×5	0.610 (−) / 0.513
	ndvi_anomaly	NDVI − local mean	0.508
Terrain	elevation (raw)	SRTM elevation, m	0.626 (+)
	slope	$\arctan\|\nabla z\|$, degrees	0.677 (+)
	aspect_sin/cos	downslope direction	0.508
	elev_rel	$z-$ mean of z in 9×9	0.527 (+)
Human	population (raw)	people / km^2	0.601 (−)
	pop_n9	mean population in 9×9	0.628 (−)

TABLE VIII
Test results. Precision (P), recall (R) and F1 at the threshold chosen on the eval split. Bold: best per column. PR AUC reported by Huot et al. [3] for their models is in Table I.

Model	PR AUC	ROC AUC	P	R	F1	R @ 1% FPR	R @ 5% FPR
Persistence	0.106	0.633	0.357	0.273	0.309	0.273	0.273
Logistic regression (raw)	0.146	0.798	0.378	0.266	0.312	0.281	0.319
Random forest (raw)	0.157	0.816	0.371	0.270	0.312	0.279	0.362
XGBoost (raw)	0.145	0.795	0.337	0.257	0.292	0.267	0.375
XGBoost (tuned, 35 features)	0.257	0.906	0.242	0.526	0.332	0.388	0.675
LightGBM (tuned, 35 features)	0.248	0.903	0.242	0.520	0.331	0.384	0.671

are left out of every metric. The raw scores are shifted by the 5:1 subsampled table and the class weighting, which is why the eval threshold is as high as 0.915; undoing that shift in closed form gives probabilities close to the observed fire rate (Section III-B). Only the ranking and the threshold affect the metrics.

C. Interpretability and physical plausibility

SHAP values [25], computed with TreeSHAP [26], of the refitted model (identical predictions, Fig. 13) confirm the gain ranking in the Feature Engineering section: the fire-neighbourhood group contributes 2.29 log-odds on average, against 0.40 for weather, 0.22 for vegetation, 0.20 for terrain and 0.13 for human features. Most effects have the expected sign (Spearman ρ between feature value and SHAP value): pixels closer to fire and with more fire nearby get higher risk (dist_to_fire −0.97, prev_fire_n3 +0.75); drier air raises it (vpd +0.72, min_rel_humidity −0.64); and so do stronger wind (vs +0.72) and fire upwind (upwind_fire_k2/k4/k8 +0.69/ + 0.61/ + 0.49, fire_wind_alignment +0.91). NDVI is mixed: bare ground and water (lowest NDVI, no fuel) get the most negative SHAP values, while over the main range greener, moister vegetation lowers risk ($\rho = -0.82$). One sign is not physical: high ERC (dry fuel) lowers the prediction ($\rho = -0.13$ overall, with the strongest negative SHAP values at the highest ERC). ERC is a coarse regional value, almost constant over a patch, so the model most likely uses it as a proxy for where and when a fire is, not as a cause.

To test whether the model pushes risk downwind, we step 1–5 pixels downwind and upwind from every day-t fire pixel and compare predictions in pixels not yet burning (Fig. 14a). The tuned model's downwind probability is higher at every distance, with the gap growing from 0.003 at 1 km to 0.029 at 5 km. The training data agrees (Fig. 14b): 5 km from the fire, the downwind-to-upwind ratio of next-day fire grows with wind speed, from 1.135 in calm wind (0–3 m/s) to 3.100 at 8 m/s or more. Our first version of this check assumed north-up patches and found the opposite in strong wind, which revealed that the arrays are stored south-up. On test the excess holds only in calm wind (1.081, against 0.782 and 0.444 at 3–8 m/s),

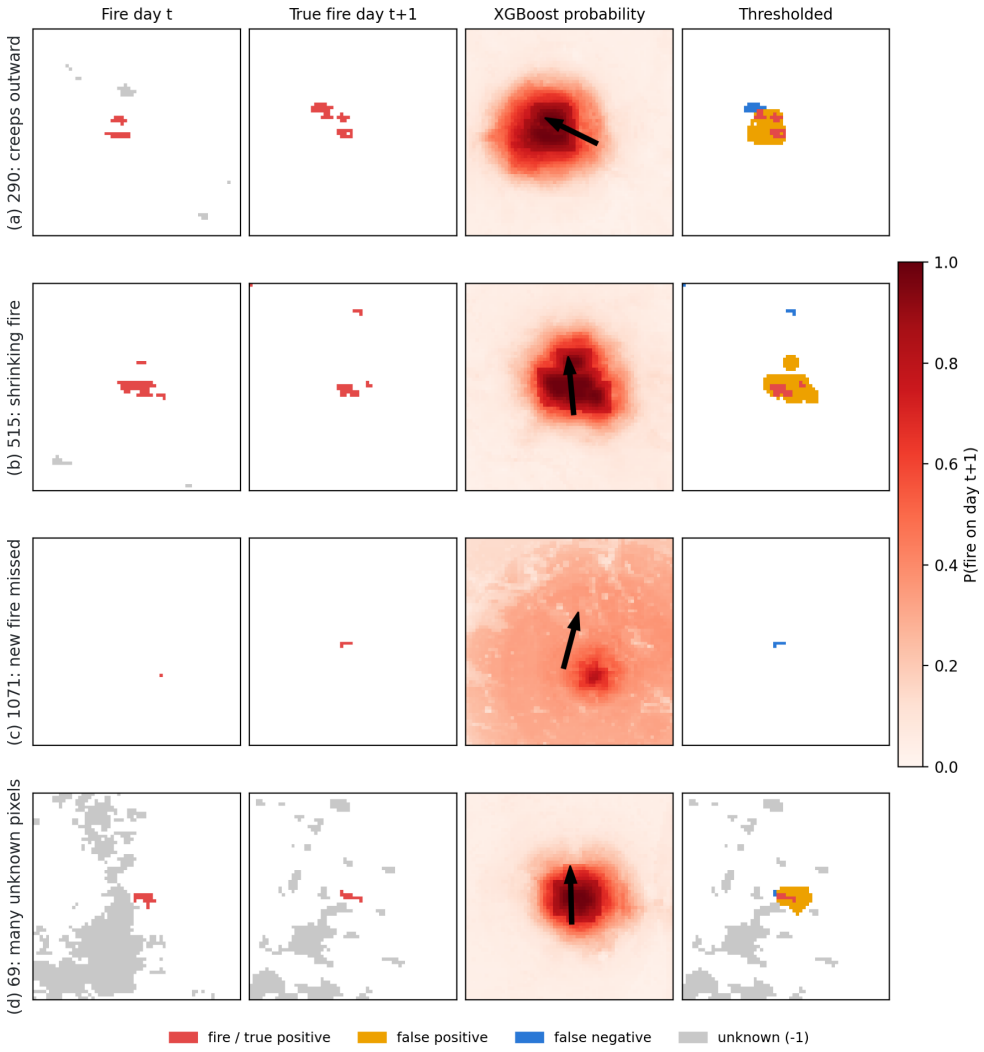


Fig. 11. Tuned XGBoost on four test patches that show its typical behaviour (all ten random patches are in Fig. 16). Columns: fire on day t, true fire on day $t+1$, predicted probability (arrow: direction the wind blows to), and prediction at the eval threshold. Drawn from the rebuilt model (Appendix B).

but these ratios rest on a few large fires (Section VII-E), so the train split is the more reliable estimate.

D. Error analysis

At the eval threshold the tuned XGBoost makes 44 409 true positives, 138 714 false positives and 39 922 false negatives. False positives form a halo around dying fires: 88.8% lie within 2 km of day-t fire, and 53.1% come from patches whose fire shrinks, although only 38.4% of test patches shrink; only 35.3% of day-t fire pixels still burn on day $t+1$. The five worst patches all hold large, scattered fires (255–468 burning pixels) that shrink the next day. Nothing in the inputs says which fires will die down (no burnt-out fuel, containment or next-day weather). False negatives come from long-range spread and sudden growth. Recall falls sharply with distance from the day-t fire (Fig. 15): 0.988 for burning pixels, 0.959 at 1 km, 0.648 at 2 km, 0.044 at 3–5 km and zero beyond 5 km, where 64.4% of all missed pixels lie. In one patch a fire grows from 8 to 260 pixels in a day and the halo covers only its centre.

E. Event-level evaluation

The test split contains duplicate patches and overlapping crops of the same fire, so pooled pixels over-weight large fires. As the dataset has no fire identifier, we link two test patches when a 16×16 elevation block of one appears exactly inside the other, and treat linked patches as one event. The 1 689 patches form only 257 events; the 10 largest hold 57% of next-day fire pixels. Scored per event (240 events with both classes), the tuned XGBoost reaches a mean PR AUC of 0.187 and a median of 0.120, and the baselines 0.074–0.101, so the ranking holds. At

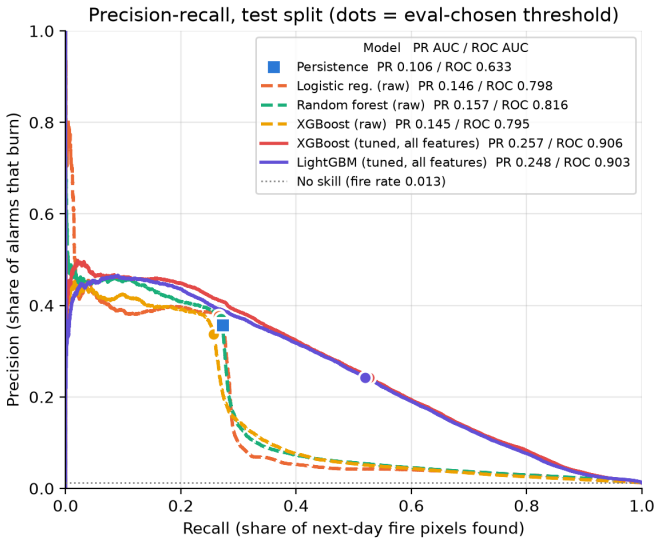


Fig. 12. Precision–recall curves on the test split. Dots: operating point at the eval-chosen threshold. Persistence outputs only 0 or 1, so it is a single point.

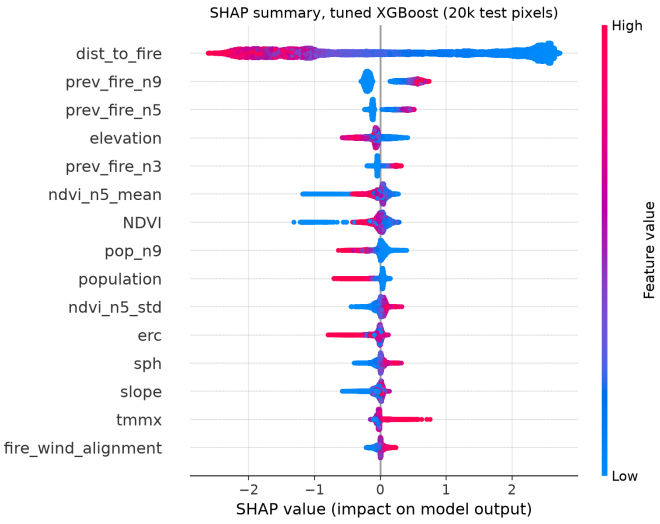


Fig. 13. SHAP values of the tuned XGBoost on 20 000 test pixels (10 000 fire, 10 000 no fire). Positive values push the prediction toward fire; colour shows the feature value.

the eval threshold, however, it finds fire in only 47.9% of events against 60.8% for persistence: a threshold that maximises pooled F1 is driven by the large fires.

F. Limitations and motivation for Phase 2

A per-pixel model with neighbourhood summaries learns a halo of roughly fixed radius around the day-t fire. It has no notion of the shape of the fire front, it cannot predict spread beyond a few kilometres, and it cannot tell a growing fire from a dying one; these are exactly its errors. The hand-made wind features carry real but small signal.

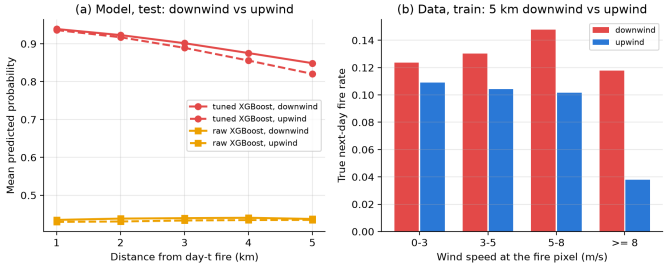


Fig. 14. Downwind check. (a) Mean predicted probability 1–5 km downwind (solid) and upwind (dashed) of day-t fire, test split. (b) True next-day fire rate 5 km downwind and upwind of day-t fire by wind speed, train split.

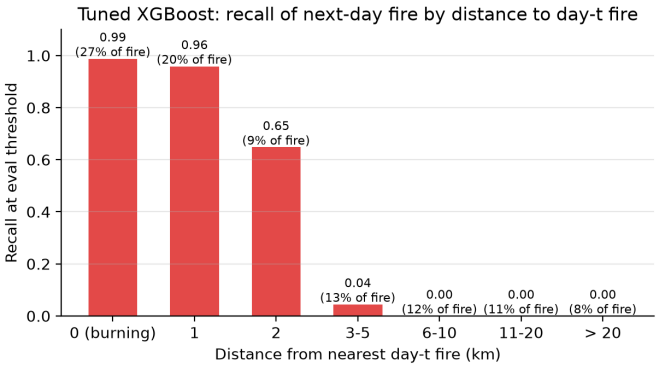


Fig. 15. Recall of next-day fire pixels by distance to the nearest day-t fire (tuned XGBoost, eval threshold). Labels give the share of all next-day fire in each band.

Phase 2 therefore moves to a CNN / U-Net that sees the whole patch, including the fire front and the wind field, as spatial input.

VIII. Conclusion

We predicted next-day wildfire spread with a per-pixel model and 23 engineered spatial features. Tuned XGBoost reaches a test PR AUC of 0.257 (95% bootstrap interval over fire events 0.19–0.33), against 0.106 for persistence. The random forest (0.225) and CNN (0.284) of Huot et al. both lie inside our interval, so our model sits between them, but we make no claim of a significant difference.

The signal is spatial. Removing the fire-neighbourhood features drops PR AUC from 0.248 to 0.048, while tuning moves it by only about 0.01. Training on 5% of the training patches gives almost the same eval PR AUC (0.204 against 0.207), so the model is limited by bias, not variance: more data or tuning will not help much.

The model finds fire at the existing front, with recall of 0.96–0.99 within 1 px of day-t fire, but recall is near zero beyond 5 km, where 64% of the missed fire pixels lie. It also raises false alarms around fires that are dying down.

Both errors come from the per-pixel design, which sees the patch only through fixed-radius summaries.

Finally, we found that the patches are stored south-up, which the dataset does not state. Every direction-dependent feature, such as fire upwind, depends on this orientation.

IX. Future Work

Phase 2 will address the limits found above.

- Whole-patch models. A U-Net [21] or CNN that sees the whole patch can learn the shape of the fire front instead of a fixed-radius halo.
- Imbalance-aware losses. Focal loss [22] or a class-weighted segmentation loss can replace negative subsampling.
- Multi-day inputs. Several days of fire history, as in WildfireSpreadTS [8], may help tell growing fires from dying ones.
- Location-disjoint splits. 30% of test patches share their location with a training patch, and these are easier for every model. A split that keeps locations or fire events apart would give a stricter test.
- Uncertain labels. Cloud and smoke pixels (-1) are now ignored; modelling them, for example by filling them from neighbouring days, would use more of the data.

Appendix A
Derivations for Section III

This appendix derives the results used in Section III. Table IX lists the notation.

TABLE IX
Notation.

Symbol	Meaning
$\mathbf{x}_i,\ y_i$	features and label (1 fire, 0 no fire) of pixel i
$F,\ f_m$	ensemble score (log-odds); the m-th tree
$p = \sigma(F)$	predicted fire probability
$T,\ w_j$	number of leaves; value of leaf j
$\lambda,\ \gamma$	L2 penalty on leaf values; penalty per leaf
$g_i,\ h_i$	gradient and Hessian of the loss for pixel i
$G_j,\ H_j$	sums of g_i and h_i over the rows in leaf j
π	true fire rate (prevalence)
$r,\ w$	share of no-fire rows kept; weight on fire rows
$P,\ N$	number of fire and no-fire pixels

A. Gradient and Hessian of the log loss

For one pixel, $l = -[\,y\log p + (1-y)\log(1-p)\,]$ with $p = \sigma(\hat{y})$. The sigmoid satisfies $\partial p/\partial\hat{y} = p(1-p)$, so by the chain rule

$$\begin{aligned} g = \frac{\partial l}{\partial \hat{y}} &= -\left[\frac{y}{p} - \frac{1-y}{1-p}\right] p(1-p) \\ &= -\big[\,y(1-p) - (1-y)p\,\big] = p - y, \end{aligned} \tag{18}$$

$$h = \frac{\partial g}{\partial \hat{y}} = \frac{\partial p}{\partial \hat{y}} = p(1-p). \tag{19}$$

So g is the prediction error, and h is largest when the model is unsure ($p = 0.5$) and goes to zero when it is confident. With weight w on the positive class, a positive row has loss $-w\log p$, and the same steps give $g = w(p-1)$ and $h = w\,p(1-p)$: both are multiplied by w, as in (9).

B. Leaf values and split gain

Substituting the second-order expansion into (2) and dropping terms that do not depend on f_m,

$$\mathcal{L}^{(m)} \approx \sum_{i=1}^{n}\Big[g_i f_m(\mathbf{x}_i) + \tfrac{1}{2}h_i f_m(\mathbf{x}_i)^2\Big] + \gamma T + \tfrac{1}{2}\lambda\sum_{j=1}^{T} w_j^2. \tag{20}$$

Every row in leaf j has $f_m(\mathbf{x}_i) = w_j$, so the sum over rows becomes a sum over leaves, which gives (5). Each leaf's term is a quadratic in w_j with minimum at $w_j^* = -G_j/(H_j+\lambda)$. Substituting it back,

$$\begin{aligned} G_j w_j^* + \tfrac{1}{2}(H_j+\lambda)\,(w_j^*)^2 &= -\frac{G_j^2}{H_j+\lambda} + \frac{1}{2}\frac{G_j^2}{H_j+\lambda} \\ &= -\frac{1}{2}\frac{G_j^2}{H_j+\lambda}, \end{aligned} \tag{21}$$

which gives $\mathcal{L}^*$ in (6). A split replaces the parent's term by the terms of its two children and adds one leaf (one more γ). The decrease in $\mathcal{L}^*$ is therefore the gain (7).

C. Probability shift from subsampling and weighting

Let $f_1(\mathbf{x})$ and $f_0(\mathbf{x})$ be the feature distributions of fire and no-fire pixels. By Bayes' rule the true probability of fire is

$$p = \frac{\pi f_1(\mathbf{x})}{\pi f_1(\mathbf{x}) + (1-\pi) f_0(\mathbf{x})}. \tag{22}$$

Keeping a share r of the negatives and weighting positives by w changes the class prior seen by the loss to $w\pi : r(1-\pi)$, but leaves f_1 and f_0 unchanged. The probability that minimises the training loss is therefore

$$p' = \frac{w\pi f_1(\mathbf{x})}{w\pi f_1(\mathbf{x}) + r(1-\pi) f_0(\mathbf{x})}. \tag{23}$$

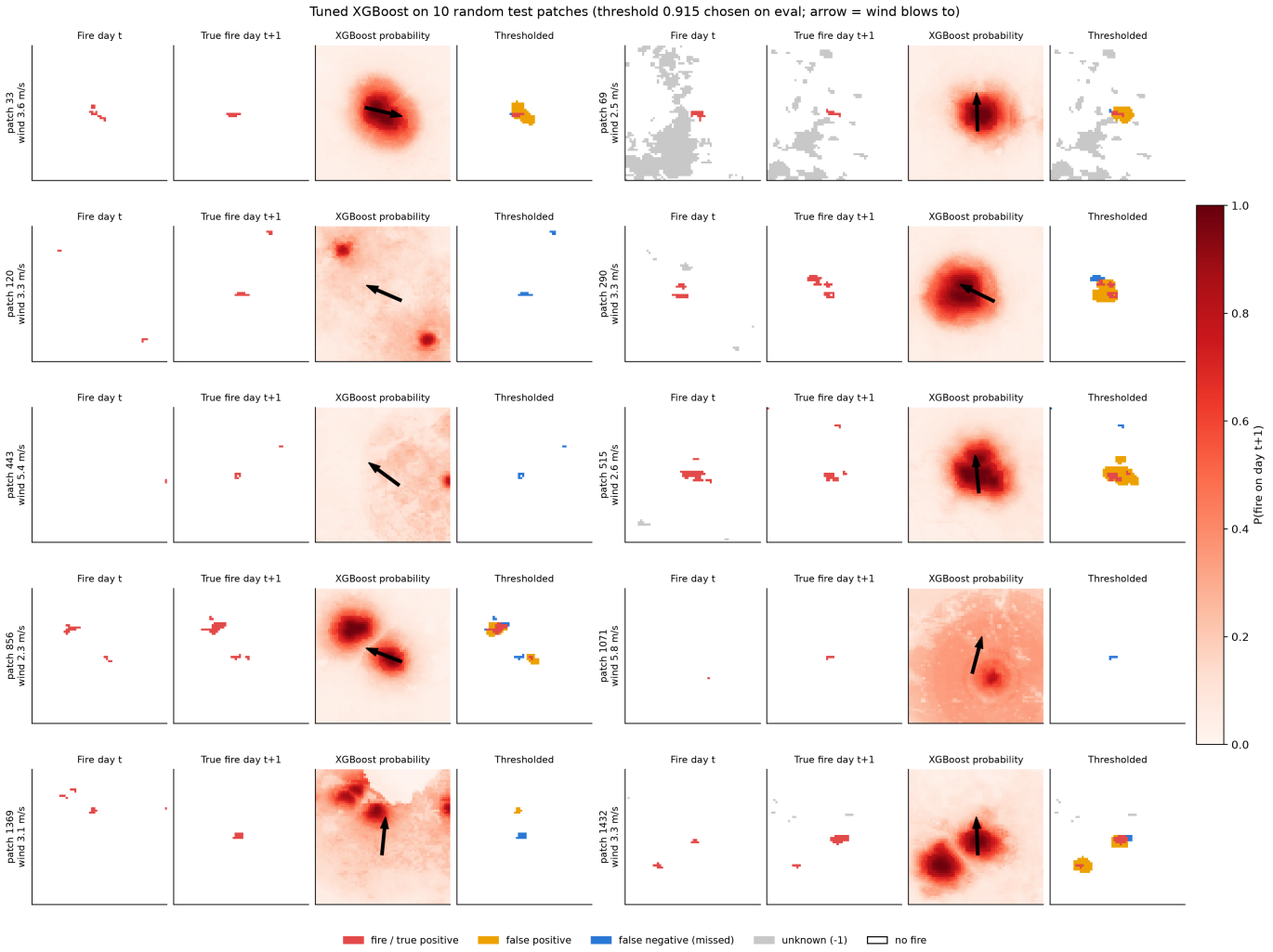


Fig. 16. Tuned XGBoost on 10 random test patches (seed 0). Per patch: fire on day t, true fire on day $t+1$, predicted probability (arrow: direction the wind blows to), and prediction at the eval threshold (red: true positive, orange: false positive, blue: false negative, grey: unknown).

Taking odds, the densities appear in the same ratio in both:

$$\frac{p'}{1-p'} = \frac{w\pi f_1(\mathbf{x})}{r(1-\pi)f_0(\mathbf{x})} = \frac{w}{r}\cdot\frac{\pi f_1(\mathbf{x})}{(1-\pi)f_0(\mathbf{x})} = \frac{w}{r}\cdot\frac{p}{1-p}. \tag{24}$$

On the log-odds scale this is a constant shift, $\operatorname{logit} p' = \operatorname{logit} p + \log(w/r)$. Solving for p gives the inverse map (11). For our values, $w/r = 90.0$, so a score of $p' = 0.915$ (odds 10.8) has true odds $10.8/90.0 = 0.12$, that is $p = 0.107$.

D. Precision as a function of prevalence

Substitute $\mathrm{TP} = \mathrm{TPR}\cdot P$ and $\mathrm{FP} = \mathrm{FPR}\cdot N$ into the precision, then divide numerator and denominator by $P + N$:

$$\begin{aligned}\mathrm{Precision} &= \frac{\mathrm{TPR}\cdot P}{\mathrm{TPR}\cdot P + \mathrm{FPR}\cdot N} \\ &= \frac{\pi\,\mathrm{TPR}}{\pi\,\mathrm{TPR} + (1-\pi)\,\mathrm{FPR}}.\end{aligned} \tag{25}$$

E. Variance of an average of trees

For B trees with variance σ^2 and pairwise correlation ρ, the variance of the sum is the sum of the B variances and the $B(B-1)$ covariances $\rho\sigma^2$:

$$\begin{aligned}\operatorname{Var}\Big(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b\Big) &= \frac{1}{B^2}\Big[B\sigma^2 + B(B-1)\,\rho\sigma^2\Big] \\ &= \frac{\sigma^2}{B} + \frac{B-1}{B}\,\rho\sigma^2 = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.\end{aligned} \tag{26}$$

Appendix B
Reproduction and uncertainty checks

Reproduction. We rebuilt the whole pipeline from the TFRecords on a Kaggle CPU machine with the repository code, the committed hyperparameters (reports/best_params.json) and seed 0. The library versions there differ slightly from requirements.txt (scikit-learn 1.6.1 against 1.9.1, LightGBM 4.6.0 against 4.7.0). Persistence, logistic regression and raw XGBoost reproduce exactly. The random forest, tuned XGBoost and LightGBM differ by at most 0.002 in test PR AUC (0.1558, 0.2576 and 0.2503 against 0.1565, 0.2565 and 0.2483). The intervals and the calibration check use these rebuilt predictions. Script and outputs are in reports/checks.

Bootstrap intervals. We resample the test set 500 times with replacement and recompute PR AUC on the pooled pixels (Table X). Pixels in one patch are not independent, so we resample whole patches. Overlapping patches of the same fire are not independent either, so we also resample whole fire events, using the 257 events of Section VII-E. The event intervals are four to five times wider, because a few large fires hold most of the fire pixels, and they are the ones we report.

TABLE X
Test PR AUC of the rebuilt models with 95% bootstrap intervals (500 resamples).

Model	PR AUC	Patches	Events
Persistence	0.1065	0.097–0.118	0.059–0.161
Logistic regr. (raw)	0.1457	0.132–0.162	0.084–0.203
Random forest (raw)	0.1558	0.144–0.169	0.096–0.212
XGBoost (raw)	0.1455	0.136–0.157	0.095–0.198
XGBoost (tuned)	0.2576	0.242–0.274	0.188–0.325
LightGBM (tuned)	0.2503	0.235–0.267	0.180–0.317

Paired differences. Two models scored on the same resample share its hard and easy fires, so their difference varies much less than either score. Over the same 500 event resamples, tuned XGBoost beats LightGBM by 0.007 (95% interval 0.001 to 0.011, higher in 99% of resamples), and beats the raw random forest by 0.102 (0.088 to 0.119). LightGBM beats the raw random forest by 0.095 (0.081 to 0.113), and the raw random forest beats persistence by 0.049 (0.035 to 0.055). The gain from the engineered features is therefore clear. The gap between the two boosting libraries is consistent but small.

Comparison with Huot et al. Their random forest (0.225) and CNN (0.284) both lie inside our event interval (0.188 to 0.325). A paired comparison on the same test pixels would be much tighter, as above, but it needs their predictions, which are not published. We therefore place our score between theirs and do not claim a significant difference in either direction.

Calibration and learning curves are discussed in Sections III-B and III-D (Figs. 5 and 7).

Appendix C
All probability maps

Fig. 16 shows the tuned XGBoost on 10 test patches drawn at random (seed 0), from which the examples in Fig. 11 are taken.

References

[1] P. L. Andrews, "The Rothermel surface fire spread model and associated developments: A comprehensive explanation," U.S. Department of Agriculture, Forest Service, Rocky Mountain Research Station, Fort Collins, CO, General Technical Report RMRS-GTR-371, Mar. 2018, 121 p.

[2] P. Jain, S. C. P. Coogan, S. G. Subramanian, M. Crowley, S. Taylor, and M. D. Flannigan, "A review of machine learning applications in wildfire science and management," Environmental Reviews, vol. 28, no. 4, pp. 478–505, 2020.

[3] F. Huot, R. L. Hu, N. Goyal, T. Sankar, M. Ihme, and Y.-F. Chen, "Next day wildfire spread: A machine learning dataset to predict wildfire spreading from remote-sensing data," IEEE Transactions on Geoscience and Remote Sensing, vol. 60, pp. 1–13, 2022.

[4] R. C. Rothermel, "A mathematical model for predicting fire spread in wildland fuels," USDA Forest Service, Intermountain Forest and Range Experiment Station, Ogden, UT, Research Paper INT-115, 1972.

[5] Y. O. Sayad, H. Mousannif, and H. Al Moatassime, "Predictive modeling of wildfires: A new dataset and machine learning approach," Fire Safety Journal, vol. 104, pp. 130–146, 2019.

[6] J. L. Hodges and B. Y. Lattimer, "Wildland fire spread modeling using convolutional neural networks," Fire Technology, vol. 55, no. 6, pp. 2115–2142, 2019.

[7] D. Radke, A. Hessler, and D. Ellsworth, "FireCast: Leveraging deep learning to predict wildfire spread," in Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19). International Joint Conferences on Artificial Intelligence Organization, 2019, pp. 4575–4581.

[8] S. Gerard, Y. Zhao, and J. Sullivan, "WildfireSpreadTS: A dataset of multi-modal time series for wildfire spread prediction," in Advances in Neural Information Processing Systems (Datasets and Benchmarks Track), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 74 515–74 529.

[9] J. Fitzgerald, E. Seefried, J. E. Yost, S. L. Pallickara, and N. Blanchard, "Paying attention to wildfire: Using U-Net with attention blocks on multimodal data for next day prediction," in Proceedings of the 25th International Conference on Multimodal Interaction (ICMI '23). Paris, France: ACM, Oct. 2023, pp. 470–480.

[10] H. Xiao, Y. Zhu, Y. Sun, G. Zhang, and Z. Gong, "Wildfire spread prediction using attention mechanisms in U2-NET," Forests, vol. 15, no. 10, p. 1711, 2024.

[11] H. S. Andrianarivony and M. A. Akhloufi, "LinU-Mamba: Visual Mamba U-Net with linear attention to predict wildfire spread," Remote Sensing, vol. 17, no. 15, p. 2715, 2025.

[12] M. Marjani and M. S. Mesgari, "The large-scale wildfire spread prediction using a multi-kernel convolutional neural network," ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences, vol. X-4/W1-2022, pp. 483–488, 2023.

[13] Y. Zhou, R. Kong, Z. Xu, L. Xu, and S. Cheng, "Comparative and interpretative analysis of CNN and transformer models in predicting wildfire spread using remote sensing data," Journal of Geophysical Research: Machine Learning and Computation, vol. 2, no. 2, p. e2024JH000409, 2025.

[14] M. Esparza, A. Ayanzadeh, A. Mousavi, and A. Mostafavi, "Modular deep learning mechanisms for auditable next-day wildfire spread prediction," 2026, arXiv preprint arXiv:2609.17763. [Online]. Available: https://arxiv.org/abs/2609.17763

[15] J. Han, J. Lee, H. Han, Y. Na, and J.-J. Lee, "FireSenseNet: A dual-branch CNN with cross-attentive feature interaction for next-day wildfire spread prediction," 2026, arXiv preprint arXiv:2604.07675. [Online]. Available: https://arxiv.org/abs/2604.07675

[16] Y. Luo, S. Rong, A. Watts, and A. E. Cetin, "U-Net with Hadamard transform and DCT latent spaces for next-day wildfire spread prediction," 2026, arXiv preprint arXiv:2602.11672. [Online]. Available: https://arxiv.org/abs/2602.11672

[17] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[18] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ser. KDD '16. ACM, 2016, pp. 785–794.

[19] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

[20] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[21] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, ser. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2015, pp. 234–241.

[22] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in 2017 IEEE International Conference on Computer Vision (ICCV). IEEE, 2017, pp. 2999–3007.

[23] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in Proceedings of the 23rd International Conference on Machine Learning (ICML '06). ACM, 2006, pp. 233–240.

[24] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, p. e0118432, 2015.

[25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf

[26] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56–67, 2020.